

บทที่ 1

ความรู้ทั่วไปเกี่ยวกับดาต้าแวร์เฮ้าส์

1.1 ประวัติความเป็นมาและความหมายของดาต้าแวร์เฮ้าส์

แนวความคิดของดาต้าแวร์เฮ้าส์ได้เกิดขึ้นมาในช่วงต้นปี 80 ซึ่งในอดีตที่ผ่านมา นั้นฐานข้อมูลหนึ่ง ๆ จะถูกใช้งานทั้งในงานโอเปอเรชันนอล และงานวิเคราะห์ข้อมูล แต่เนื่องจากฐานข้อมูลนั้นถูกออกแบบมาโดยยึดตามงานประเภททรานแซกชัน ซึ่งจะมีการเปลี่ยนแปลงข้อมูลอยู่เสมอ ๆ ทำให้งานในการวิเคราะห์ข้อมูลทำได้ยากขึ้น ดังนั้นในปัจจุบันจึงมีการแยกฐานข้อมูลที่ใช้ในการวิเคราะห์ออกมาจากฐานข้อมูลทรานแซกชัน โดยที่ฐานข้อมูลที่ถูกแยกออกมาจะเป็นฐานข้อมูลที่เน้นการคิวรีข้อมูลขึ้นมาเพื่อใช้ในการตอบคำถาม นอกจากนี้ยังเป็นการเก็บข้อมูลบนพื้นฐานของการเปลี่ยนแปลงไปตามเวลาอีกด้วย นั่นคือข้อมูลเก่าที่มีความสำคัญต่อการวิเคราะห์ข้อมูลนั้นจะไม่ถูกลบไปจากฐานข้อมูลนี้ ซึ่งฐานข้อมูลที่ถูกแยกออกมาเราจะเรียกว่า “Data Warehousing Database”

ดาต้าแวร์เฮ้าส์เป็นฐานข้อมูลที่ทำกรคัดแยกข้อมูลจากฐานข้อมูลโอเปอเรชันนอล เพื่อให้อยู่ในรูปแบบของข้อมูลที่เหมาะสมต่อการวิเคราะห์ข้อมูล และการตอบคำถาม โดยสำหรับรูปแบบของข้อมูลที่เหมาะสมเราอาจจะสร้างมาในรูปแบบของฐานข้อมูลเชิงสัมพันธ์(Relational Database) หรือจะเป็นฐานข้อมูลเชิงวัตถุ(Object Oriented Database) หรือจะเป็นฐานข้อมูลเชิงวัตถุสัมพันธ์(Object –Relational Database) ก็ได้ สำหรับข้อดีข้อเสียของแต่ละแบบเราจะกล่าวถึงในบทที่ 2

1.2 ข้อดีและข้อเสียของดาต้าแวร์เฮ้าส์

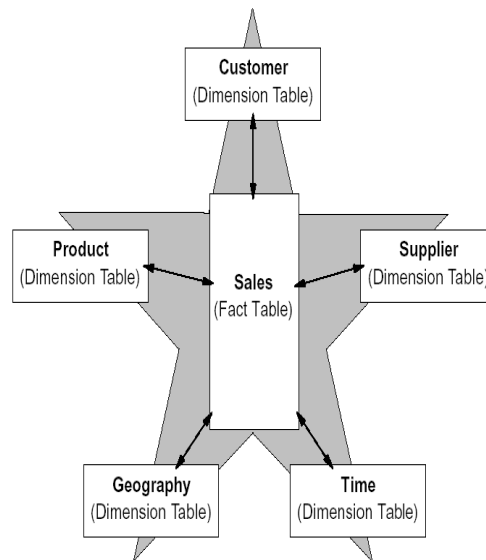
Furlow ได้อธิบายถึงข้อดี 5 ประการของดาต้าแวร์เฮ้าส์ซึ่ง ข้อแรก ดาต้าแวร์เฮ้าส์ซึ่งจะแสดงรวบรวมข้อมูลจากหลาย ๆ ที่ภายในองค์กรเข้ามาเก็บไว้ที่เดียว เพื่อให้ง่ายต่อการตอบคำถาม ข้อสองคือง่ายต่อการเข้าถึงข้อมูล ซึ่งจะนำไปสู่การวิเคราะห์ที่รวดเร็วและถูกต้อง ข้อสามคือสามารถเพิ่มประสิทธิภาพของระบบโอเปอเรชันนอล เนื่องจากมีการแยกฐานข้อมูลออกเป็นดาต้าแวร์เฮ้าส์ ทำให้ไม่มีการดึงข้อมูลไปใช้ในการวิเคราะห์จากโอเปอเรชันนอลดาต้าเบส ข้อที่สี่คือ ดาต้าแวร์เฮ้าส์มีความยืดหยุ่นมากกว่าโอเปอเรชันนอล สามารถเปลี่ยนแปลงโครงสร้างภายในได้ ข้อสุดท้ายคือดาต้าแวร์เฮ้าส์ทำให้องค์กรสามารถจัดการกับข้อมูลได้ดีขึ้นทำให้เป็นผลดีในการแข่งขันทางธุรกิจที่ต้องการความรวดเร็วในการตัดสินใจกับข้อมูลต่าง ๆ

นอกจากนี้ Furlow ยังได้กล่าวถึงข้อดีของดาต้าแวร์เฮ้าส์ คือ ความซับซ้อนในตัวดาต้าแวร์เฮ้าส์ และความยุ่งยากในการพัฒนาตัวระบบ การสร้างดาต้าแวร์เฮ้าส์หนึ่ง ๆ จะทำให้เสียเงินและเวลาเป็นอย่างมาก

1.3 รูปแบบของดาต้าแวร์เฮ้าส์

สำหรับการออกแบบดาต้าแวร์เฮ้าส์นั้นมักจะอยู่ในรูปแบบของมัลติไดเมนชันนอลดาต้าเบส (Multidimensional Database) ก็จะแบ่งข้อมูลออกเป็น 2 ประเภทคือ ไดเมนชัน ข้อมูลในตารางไดเมนชันนั้นจะเป็นข้อมูลที่เป็นส่วนประกอบของการวิเคราะห์แต่ไม่ใช่ข้อมูลที่เราสนใจ อีกประเภทหนึ่งของข้อมูลก็คือ แฟก สำหรับแฟกนั้นจะเป็นข้อมูลที่เราสนใจนำมาวิเคราะห์เช่น ยอดการขายสินค้าในแต่ละไตรมาส แต่ละเดือน หรือแต่ละปี เป็นต้น สำหรับการออกแบบนั้นเราจะใช้เทคนิคการออกแบบที่เรียกว่า Star Schema คือจะมีลักษณะของไดอะแกรมเป็นรูปดาว มี Fact Table อยู่ตรงกลาง มี Dimension Table อยู่โดยรอบ และใน Fact Table จะมี

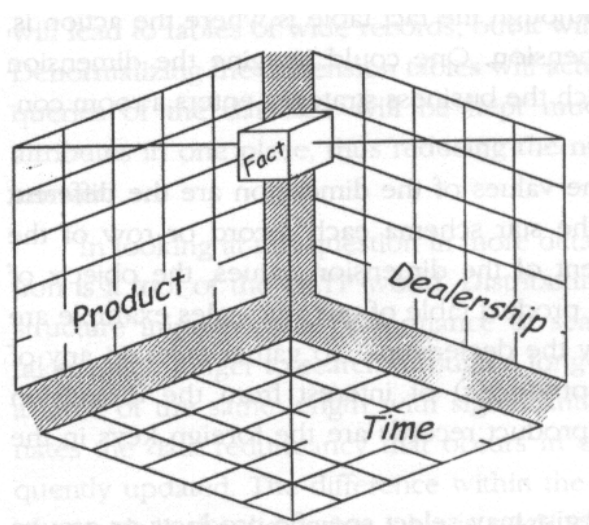
Primary Key เป็น Combine Key ที่เป็นการรวมเอา Primary Key ของ Dimension Table ที่อยู่โดยรอบมาประกอบกัน ดังแสดงในรูปที่ 1.1



รูปที่ 1.1 แสดงเทคนิคการออกแบบโดยใช้ Star Schema

นอกจากนี้ค่าตัวแปรเหล่านี้ยังมีคุณสมบัติที่ขาดไม่ได้ นั่นคือการสร้างค่าตัวแปรเหล่านี้ขึ้นมานั้นจะอยู่บนพื้นฐานของการตอบคำถามเกี่ยวกับเวลา เช่น อยากรู้ยอดขายรถยนต์ในไตรมาสที่ 3 ของปี 2000 ดังนั้นไคเมนชันของเวลา(Time Dimension) จึงเป็นไคเมนชันที่ควรจะมีในทุก ๆ ค่าตัวแปรเหล่านี้

ค่าตัวแปรเหล่านี้ในบางครั้งมักจะแสดงให้เห็นในรูปของลูกบาศก์ โดยที่แต่ละไคเมนชันก็จะแสดงได้บนด้านต่าง ๆ ของลูกบาศก์ ความสัมพันธ์ของไคเมนชันก็แสดงได้โดยพิคัดของลูกบาศก์ และที่ทุก ๆ ไคเมนชันขึ้นมาเราก็จะได้ Fact มา ดังแสดงในรูป 1.2



รูปที่ 1.2 แสดงให้เห็นถึงการนำ Cube มาแสดงความสัมพันธ์แบบ Multidimensional

บทที่ 2

ความรู้เบื้องต้นกับฐานข้อมูลเชิงวัตถุสัมพันธ์

ในปัจจุบันเมื่อความต้องการทางด้านธุรกิจสูงขึ้น จึงทำให้การออกแบบฐานข้อมูลในแบบที่ผ่านมานั้น ไม่สามารถรองรับความต้องการเพื่อทันต่อการเปลี่ยนแปลงความต้องการในการใช้งานที่เพิ่มขึ้นได้ การออกแบบฐานข้อมูลเชิงวัตถุสัมพันธ์จึงได้ถูกพัฒนาขึ้นมาเพื่อให้รองรับกับความต้องการที่เปลี่ยนไป

ในบทนี้จะกล่าวถึงความหมาย และความแตกต่างระหว่างฐานข้อมูลทั้ง 3 แบบ ซึ่งได้แก่ Relational Database , Object – Oriented Database และ Object – Relational Database

2.1 Relational Database

ฐานข้อมูลเชิงสัมพันธ์นั้นจะประกอบด้วยแถวและคอลัมน์ เป็นความสัมพันธ์ที่อยู่ในรูปแบบของตาราง ดังรูปที่ 2.1

Employee ID	Employee Name	Employee Address	Employee Age	
1085				
1086				
1087				
1088				
1089				
1090				
1091				
1092				
1093				
1094				

รูปที่ 2.1 แสดงความสัมพันธ์ระหว่างคอลัมน์และแถวในรูปแบบของตาราง

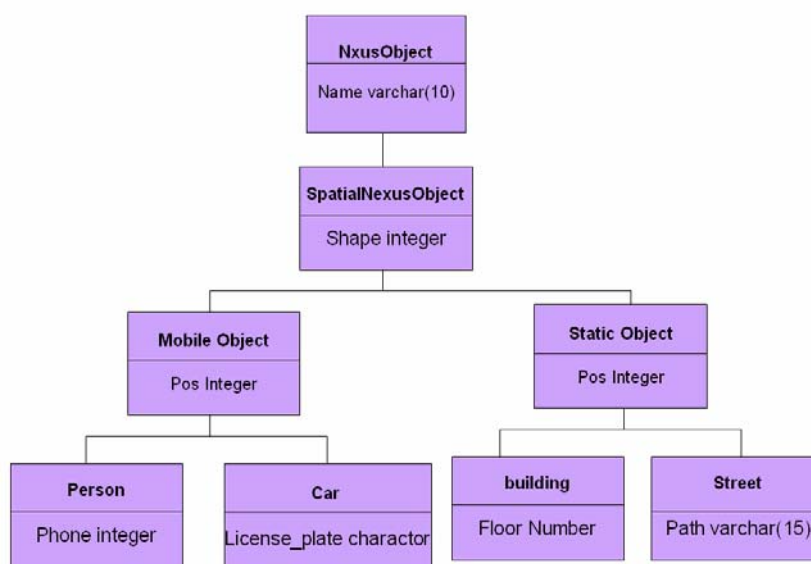
ภาษาที่ใช้ในฐานข้อมูลประเภทนี้นั้นเป็นภาษาที่เรียกว่า Structure Query Language(SQL) เป็นภาษาในการสร้างฐานข้อมูล(DDL : Data Definition Language) และภาษาในการเปลี่ยนแปลงข้อมูลในฐานข้อมูล(DML : Data Manipulation Language) ภาษา SQL นี้จะรวมคำสั่งที่ใช้ในการกำหนดชนิดของข้อมูล คำสั่งในการเปลี่ยนแปลงข้อมูลในฐานข้อมูล คำสั่งในการเรียกข้อมูลขึ้นมาตอบคำถาม คำสั่งที่ใช้ในการกำหนดเงื่อนไข คำสั่งในการสร้างตาราง เป็นต้น

แต่เนื่องจากภาษา SQL ที่ใช้ใน Relational Database นั้นเป็น SQL92 ซึ่งมีข้อจำกัดอยู่ที่ SQL92 จะจำกัดประเภทการใช้ตัวแปรเพียง Build-in Function จำพวกตัวเลข หรือ String เท่านั้น แต่ในปัจจุบันข้อมูลบางส่วนมีลักษณะเป็น Complex Type เช่น Geographic Points , Text และ Digital Signal มากกว่าจึงทำให้ Relational Database ไม่เหมาะสมกับการใช้งานข้อมูลในลักษณะดังกล่าว

นอกจากนี้ตารางใน Relational นั้นเป็น Flat Table ซึ่งทำให้ไม่สามารถรองรับข้อมูลประเภท Nested Structure จำพวก Set ,Array หรือ Nested Table

2.2 Object – Oriented Database

ความมุ่งหวังที่จะทำให้การนำเสนอโครงสร้างของข้อมูลที่ซับซ้อนให้ง่ายขึ้นนั้น ทำให้เกิดแนวความคิดในการออกแบบฐานข้อมูลเชิงวัตถุ(Object – Oriented Database) และแนวความคิดของ Abstract Data Type ก็ได้มีการพัฒนาขึ้นมา เพื่อให้มีซ่อน โครงสร้างของข้อมูลเอาไว้และให้มีโอเปอเรชันภายนอกมาทำการจัดการกับข้อมูลในโครงสร้างขึ้น ซึ่งต่อมาก็กลายเป็นความสามารถของ Object ในการ Encapsulation นอกจากนี้ก็ยังมีความสามารถของ Object ในการ Inheritance และ Polymorphism Inheritance เป็นความสามารถของ object ในการรับเอาคุณสมบัติของ Object ต้นแบบมาใช้ได้ Polymorphism เป็นความสามารถที่อนุญาตโอเปอเรเตอร์ตัวเดียวกันสามารถมีฟังก์ชันทำงานที่แตกต่างกันได้ สำหรับฐานข้อมูลเชิงวัตถุนั้นจะมีการสร้าง Object Identify ซึ่งทำหน้าที่คล้ายกับ Primary Key เพื่อใช้เป็นตัวเข้าถึง Object นั้น ๆ object Identify (OID) นั้นจะถูกสร้างขึ้นทุกครั้งที่มีการเพิ่มอ็อบเจกต์เข้าไปในดาต้าเบส โมเดลการออกแบบจะแสดงให้เห็นถึงความสัมพันธ์สองประการคือ ประการแรกจะแสดงความสัมพันธ์ภายในอ็อบเจกต์ ประการที่สองจะแสดงความสัมพันธ์แบบ Inheritance ดังรูป



รูปที่ 2.2 แสดงความโมเดลการออกแบบฐานข้อมูลเชิงวัตถุ

นอกจากนี้ภาษาจำพวก Object – Oriented Programing ยังมีผลต่อการลดขนาดของ Code Database อีกด้วย เพราะในบางฟังก์ชันไม่จำเป็นต้องแปลฟังก์ชันการทำงานเป็นภาษาทางด้านดาต้าเบส

แต่อย่างไรก็ตามการออกแบบฐานข้อมูลเชิงวัตถุนั้นก็ยังส่งผลให้เกิดปัญหาการลดประสิทธิภาพในการเข้าถึงข้อมูลต่ำ การ Query Data ทำได้ยากและมีความซับซ้อนมาก ดังนั้นฐานข้อมูลเชิงวัตถุนั้นก็จึงไม่เหมาะกับฐานข้อมูลที่มีขนาดใหญ่

2.3 Object – Relational Database

จุดประสงค์ของการสร้างฐานข้อมูลเชิงวัตถุสัมพันธ์(Object – relational Database) คือการปรับปรุงวิธีการออกแบบฐานข้อมูลเพื่อให้เข้ากับความต้องการในการใช้งานในปัจจุบัน โดยการนำเอาข้อดีของฐานข้อมูลเชิงสัมพันธ์ และฐานข้อมูลเชิงวัตถุมารวมกัน

Stonebraker และ Brown 1999 ได้กล่าวถึงคุณลักษณะที่สำคัญของฐานข้อมูลเชิงวัตถุสัมพันธ์ว่า 1. สนับสนุนการสร้างชนิดของข้อมูล 2. สนับสนุนโครงสร้างข้อมูลที่มีความซ้ำซ้อน 3. สนับสนุนความสามารถในการ Inheritance 4. สนับสนุน Data Integrity

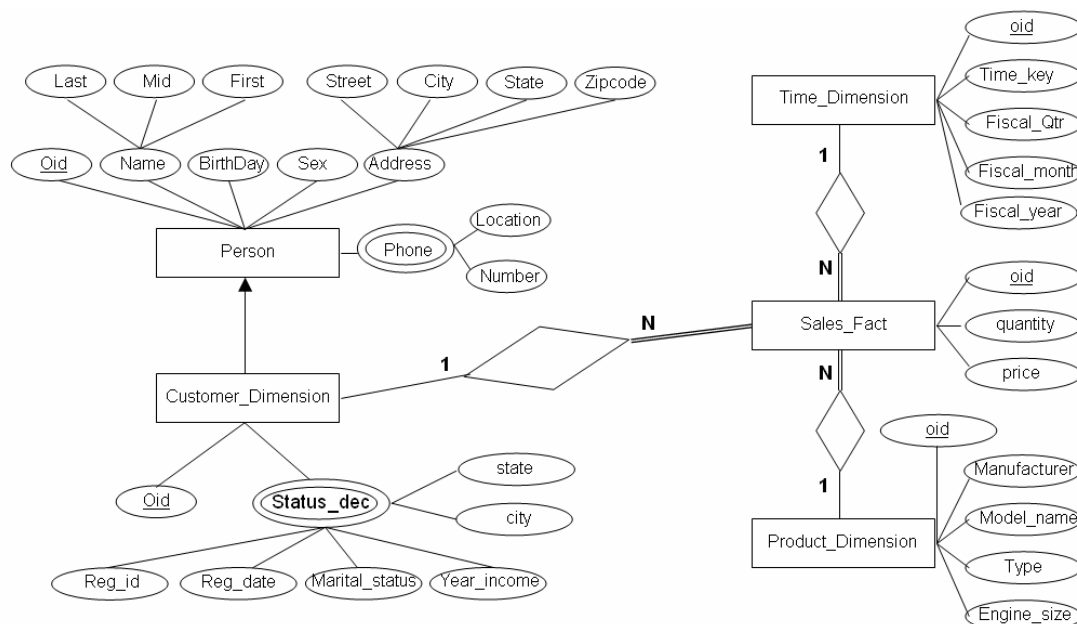
ฐานข้อมูลแบบวัตถุสัมพันธ์จะยอมให้ผู้ใช้สามารถประกาศชนิดของข้อมูล ฟังก์ชันการทำงาน และโอเปอเรเตอร์เองได้ โดยภาษาที่ใช้ในฐานข้อมูลเชิงวัตถุสัมพันธ์นั้นจะเป็น SQL99 หรือ Object – SQL หรือ SQL3 ซึ่งเป็นมาตรฐานของภาษาที่ได้รับการยอมรับในปี 1999

ตัวอย่างโครงสร้างข้อมูลที่ฐานข้อมูลเชิงวัตถุสัมพันธ์รองรับ ได้แก่ Employee(ID,Name(first,mid,last), Sex,Birthday,Address(street,city,state,zipcode),Phones(location,phone_num),DepartName,Picture)

ซึ่งในตัวอย่างข้างต้นจะเห็นได้ว่า Name และ Address เป็น Composite Attribute คือหมายความว่า Attribute นั้นประกอบด้วย Attribute หลาย ๆ Attribute ส่วน Phones นอกจากจะเป็น Composite Attribute แล้วยังเป็น Multivalues Attribute อีกด้วย นั่นหมายความว่า มีหลายข้อมูลอยู่ใน Attribute เดียว เช่นมี HomePhone, WorkPhone,MobilePhone เป็นต้น และ Picture ก็เป็น Attribute ประเภทรูปภาพ แต่ก็มีบาง Attribute เช่นกันที่ฐานข้อมูลเชิงสัมพันธ์สนับสนุน

แต่ถึงฐานข้อมูลเชิงวัตถุสัมพันธ์จะมีข้อดีหลายอย่างแต่ก็ยังมียกเสี้ยนเช่นกันคือ ฐานข้อมูลเชิงวัตถุสัมพันธ์นั้นจะไม่เหมาะกับการทำงานประเภท Web Application ซึ่งต้องการฐานข้อมูลที่มีความเร็วสูง แต่ถ้าเทียบกับข้อดีข้ออื่น ๆ แล้วฐานข้อมูลเชิงวัตถุสัมพันธ์ก็จะมีแนวโน้มที่จะยึดครองตลาดฐานข้อมูลได้ในอนาคต

โมเดลการออกแบบของฐานข้อมูลประเภทนี้นั้นจะแสดงได้โดยการนำคุณสมบัติของฐานข้อมูลเชิงวัตถุสัมพันธ์และคุณสมบัติของฐานข้อมูลเชิงวัตถุมารวมกัน ดังตัวอย่างที่แสดง



รูปที่ 2.3 โมเดลแสดงความสัมพันธ์ของ Object Relational Database

จากภาพเป็นการออกแบบฐานข้อมูลโดยใช้ ER Diagram ซึ่งใช้ในการออกแบบฐานข้อมูลเชิงสัมพันธ์มาใช้ และจะเห็นว่า Customer_dimension มีการ Inherit คุณสมบัติมาจาก Person นอกจากนี้ยังมี Composite Attribute และ Multivalues Attribute อีกด้วย

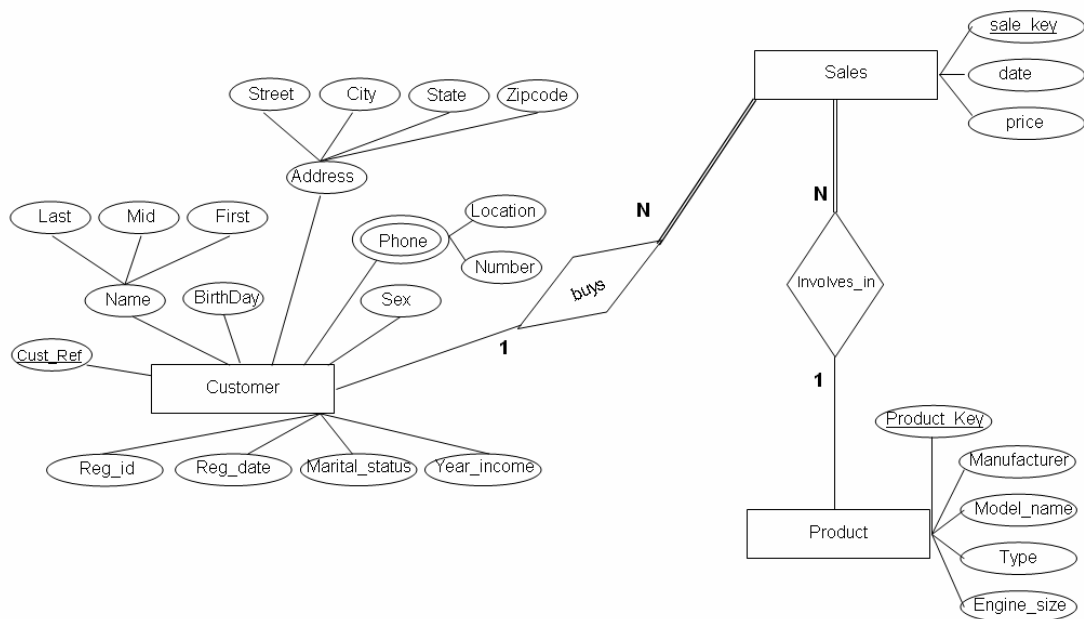
บทที่ 3

การออกแบบฐานข้อมูลดาต้าแวร์เฮ้าส์

จากข้อดีของฐานข้อมูลเชิงวัตถุสัมพันธ์ในบทที่ผ่านมา ซึ่งถ้าหากพิจารณาถึงความต้องการในการใช้งานฐานข้อมูลในปัจจุบันแล้ว การออกแบบดาต้าแวร์เฮ้าส์โดยใช้วิธีการออกแบบฐานข้อมูลเชิงวัตถุสัมพันธ์จึงเป็นทางเลือกที่เหมาะสมที่สุด ในบทนี้จะกล่าวถึงโมเดลการออกแบบซึ่งจะแสดงให้เห็นในรูปแบบของการออกแบบฐานข้อมูลเชิงวัตถุสัมพันธ์ และเพื่อให้ง่ายต่อการเข้าใจ จึงนำวิธีการออกแบบดาต้าแวร์เฮ้าส์ในรูปแบบดาว(Star Schema)มาใช้

3.1 การออกแบบฐานข้อมูลโดยใช้ฐานข้อมูลเชิงวัตถุสัมพันธ์กับทรานแซกชันดาต้าเบส

ในการศึกษาการประยุกต์นำเอาการออกแบบคลังข้อมูลเชิงวัตถุสัมพันธ์(Object – Relational Information Warehouse) นั้น ในกรณีนี้เราจะนำเอากระบวนการขายรถมาเป็นตัวอย่างการออกแบบ โดยที่คนขายรถนั้นจะเก็บข้อมูลการขายลงในฐานข้อมูลของการขายรถ สำหรับการศึกษาเราจะกล่าวถึงเพียงโมเดลออกแบบเพียงเท่านั้น แต่จะไม่กล่าวถึงวิธีการออกแบบโมเดลดังกล่าว และการออกแบบฐานข้อมูลดังกล่าวเพื่อให้เข้าใจง่ายจะใช้ ER Diagram ในการออกแบบ



รูปที่ 3.1 ER Diagram แสดง Model การออกแบบทรานแซกชันดาต้าเบส
โดยใช้แนวคิดของฐานข้อมูลเชิงวัตถุสัมพันธ์

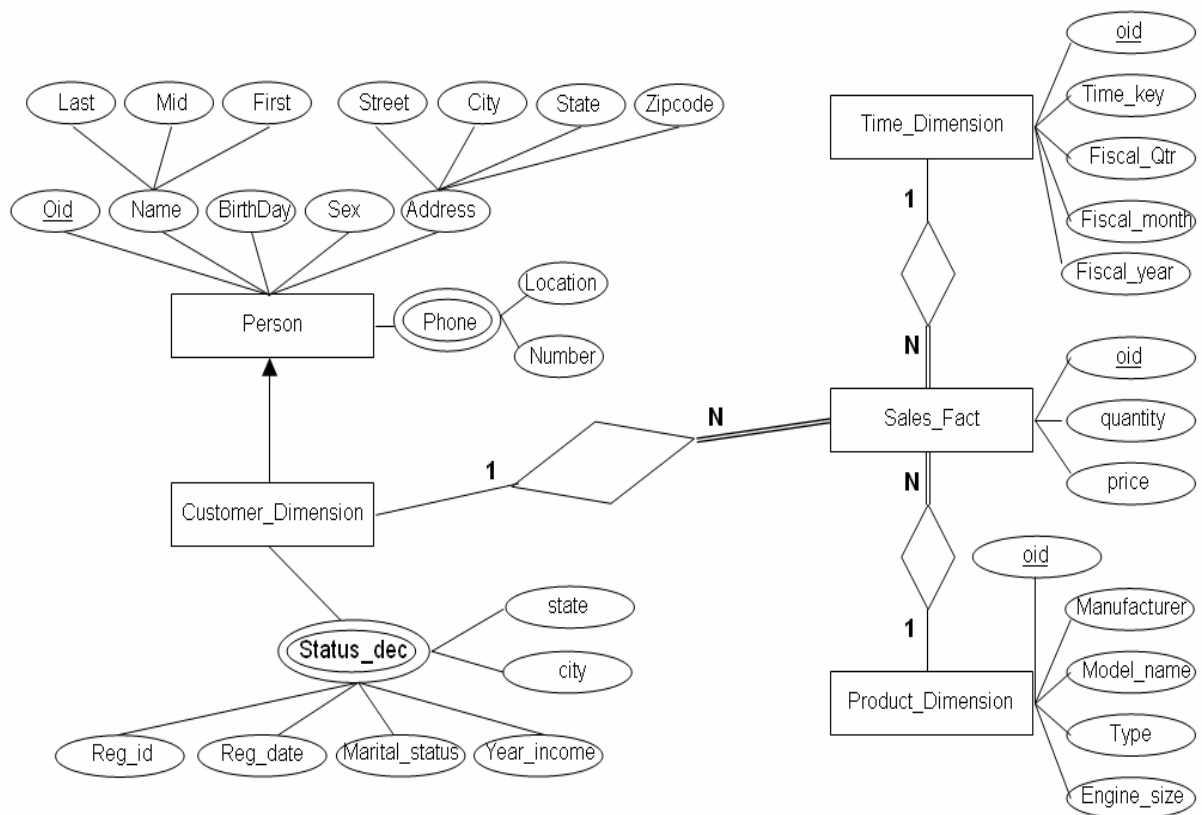
จากภาพจะเห็นว่า Model จะประกอบไปด้วย 3 Entity Type นั่นคือ 1. Customer Entity Type จะคอยดูแลข้อมูลต่างๆ ที่เกี่ยวข้องกับลูกค้าที่มาซื้อรถ 2. Sale Entity Type จะคอยดูแลข้อมูลที่เกี่ยวข้องกับการซื้อขายรถยนต์ 3. Product Entity Type จะคอยดูแลข้อมูลเกี่ยวกับสินค้า ซึ่งในที่นี้หมายถึงรถที่ขาย

ใน Diagram นี้ยังประกอบไปด้วย Attribute ประเภทต่าง ๆ ดังนี้ 1. Simple Attribute เป็นแอททริบิวต์ทั่วไป อาทิเช่น Customer.Sex, Customer.Birthday, Customer.Marital_Status, Customer.year_income, Customer.Reg_id, Product.Manufacturer, Product.Model_name, Product.Type, Product.size, Sale.quantity, Sale.price เป็นต้น 2. Composite Attribute ได้แก่ Customer.name และ Customer.Address 3. Multivalues Attribute ได้แก่ Customer.phone เป็นต้น

นอกจากนี้ Diagram ยังแสดงให้เห็นถึงความสัมพันธ์ระหว่าง Entity Type ได้แก่ Buy เป็นความสัมพันธ์ระหว่าง Customer Entity กับ Sale Entity โดยเป็นความสัมพันธ์แบบ one- to-many หมายความว่าคน 1 คนสามารถมีรายการซื้อขายได้หลายรายการ อีกความสัมพันธ์หนึ่งที่ Diagram แสดงให้เห็นคือ involve_in เป็นความสัมพันธ์ระหว่าง Sales กับ Product และเป็นความสัมพันธ์แบบ one – to – many หมายความว่า 1 คันสามารถมีรายการการซื้อได้มากกว่า 1 รายการเช่นเดียวกัน

3.2 การออกแบบดาต้าแวร์เฮ้าส์โดยใช้เทคนิคฐานข้อมูลเชิงวัตถุสัมพันธ์

สำหรับในหัวข้อนี้ เราจะสร้าง ER Diagram ของ Warehouse Database กัน สำหรับการสร้าง ER Diagram ของแวร์เฮ้าส์ดาต้าเบสนั้นจะมีพื้นฐานอยู่บนการออกแบบแบบ Star Schema ซึ่งจะประกอบไปด้วยโครงสร้าง 2 อย่างคือ Dimension และ Fact สำหรับ Fact Table นั้น จะเป็นตารางที่อยู่ตรงกลางที่แสดงจะเก็บข้อมูลที่เรานสนใจ ซึ่งจะแสดงโดยความสัมพันธ์ของ Dimension Table ที่อยู่โดยรอบ สำหรับ Dimension Table นั้นจะอยู่โดยรอบของ Fact Table และ Primary Key ของ Dimension Table นั้นก็จะไปเกาะเป็น Combine Key Attribute ของ Fact Table ดังรูปที่ 3.2



รูปที่ 3.2 ER Diagram แสดงการออกแบบฐานข้อมูลของดาต้าแวร์เฮ้าส์

จากรูปจะเห็นว่า มี 3 Dimension Table และ 1 Fact Table ประกอบด้วย

Dimension Table

ตาราง	การนำไปใช้งาน
Dimension Table	
Customer Dimension	เก็บข้อมูลเกี่ยวกับลูกค้า Inherit มาจาก Person
Time Dimension	เก็บข้อมูลเกี่ยวกับเวลา
Product Dimension	เก็บข้อมูลเกี่ยวกับสินค้าที่ขาย
Fact Table	
Sale Fact Table	เก็บข้อมูลเกี่ยวกับการขาย ซึ่งนำไปเป็นข้อมูลประกอบวิเคราะห์

ตารางที่ 3-1 ตารางแสดงความหมายของ Dimension และ Fact
ในแผนภาพ ER Diagram

สำหรับการออกแบบโดยใช้เทคนิคเชิงวัตถุสัมพันธ์นั้นไม่จำเป็นต้องมี Key Attribute แต่เราจะใช้คุณสมบัติของ OID แทน โดยที่ OID จะเป็นตัวชี้ไปยังแถวต่าง ๆ ในตาราง

บทที่ 4

การสร้างและการเปลี่ยนแปลงข้อมูลในดาต้าแวร์เฮ้าส์

จากบทที่ผ่านมา เราได้ศึกษาถึงวิธีการออกแบบดาต้าแวร์เฮ้าส์โดยใช้หลักการของระบบฐานข้อมูลเชิงวัตถุสัมพันธ์ สำหรับในบทนี้เราจะกล่าวถึงการพัฒนาระบบเพื่อนำไปใช้งานจริง

สำหรับเครื่องมือที่ใช้ในการพัฒนาระบบ เราจะใช้ IBM DB2 ซึ่งเป็นเครื่องมือที่ถูกพัฒนาโดยบริษัท ไอบีเอ็ม (ประเทศไทย) จำกัด ซึ่งมีอุปกรณ์ที่จำเป็นต่อการพัฒนาระบบฐานข้อมูลเชิงวัตถุสัมพันธ์

สำหรับในบทนี้เราจะแบ่งออกเป็น 2 ส่วนคือส่วนแรกคือส่วนที่ใช้ในการสร้างดาต้าแวร์เฮ้าส์ (DDL : Data Definition Language) ส่วนที่สองจะเป็นส่วนที่ใช้ในการปรับปรุงและเปลี่ยนแปลงข้อมูลในดาต้าแวร์เฮ้าส์

4.1 ภาษาที่ใช้ในการสร้างดาต้าแวร์เฮ้าส์(Data Definition Language)

สำหรับการสร้างฐานข้อมูลของดาต้าแวร์เฮ้าส์นั้น เราจะสร้างจาก ER Diagram ในบทที่ผ่านมา และเพื่อให้่ายเราจะทำการแปลง Entity และ Attribute จาก ER Diagram มายังตาราง

สำหรับในหัวข้อนี้เราจะเริ่มจากสร้างตารางไคเมนชั้นง่าย ๆ ได้แก่ ไคเมนชั้นของเวลา(Time Dimension)และ ไคเมนชั้นของสินค้า(Product Dimension)ก่อน หลังจากนั้นก็จะทำการสร้างตารางไคเมนชั้นที่ซับซ้อนกว่าคือ ไคเมนชั้นของลูกค้า(Customer Dimension) และสุดท้ายเราก็จะสร้างตารางแฟค(Fact Table)

4.1.1 Time Dimension Table

สำหรับการสร้าง Time Dimension Table นั้นก่อนอื่นเราจำเป็นต้องสร้าง Data Type ขึ้นมาก่อน สำหรับการสร้าง Data Type ขึ้นมานั้นเราจะใช้คำสั่ง Create Type ในภาษา SQL99 ดังนี้

```
CREATE TYPE time_dimension_t AS
(
    time_key          CHAR(10),
    fiscal_year       NUM(4),
    fiscal_qtr        CHAR(10),
    fiscal_month      CHAR(10)
)
MODE DB2SQL
REF USING INTEGER
```

ต่อจากนั้นก็จะทำการสร้าง Time Dimension ขึ้นมาโดยใช้คำสั่ง

```
CREATE TABLE time_dim_tab OF time_dimension_t
```

```
(  
    REF IS OID USER GENERATED,  
    time_key WITH OPTION NOT NULL,  
    fiscal_qtr WITH OPTION NOT NULL,  
    fiscal_month WITH OPTION NOT NULL,  
    fiscal_year WITH OPTION NOT NULL  
)
```

4.1.2 Product Dimension Table

คล้ายกับตารางไคเมนชั้นของเวลา สำหรับตารางไคเมนชั้นของสินค้า นั้นจะใช้ชื่อว่า Prod_dim_tab โดยสำหรับตารางนี้จะประกอบด้วย ซิมเปิลแอททริบิวต์(Simple Attribute) ได้แก่ oid ซึ่งทำหน้าที่เป็นออปเจ็กไคเดนทิไฟเลอร์ ที่ชี้ไปยังแต่ละแถวของตาราง Prod_dim_tab นอกจากนี้ยังมีแอททริบิวต์อื่น ๆ ได้แก่ model_name , manufacturer, type, engine_size และก่อนที่จะสร้าง Prod_dim_tab นั้น เราจำเป็นต้องสร้าง prod_dimension_t ซึ่งเป็นโครงสร้างของข้อมูลขึ้นมาก่อน โดยสำหรับคำสั่งเอสคิวแอลที่ใช้ในการสร้าง Prod_dimention_t คือ

```
CREATE TYPE prod_dimension_t AS
```

```
(  
    model_name    VARCHAR(30),  
    manufacturer   VARCHAR(25),  
    type           VARCHAR(50),  
    engine_size    CHAR(6)  
)
```

```
MODE DB2SQL
```

```
REF USING INTEGER
```

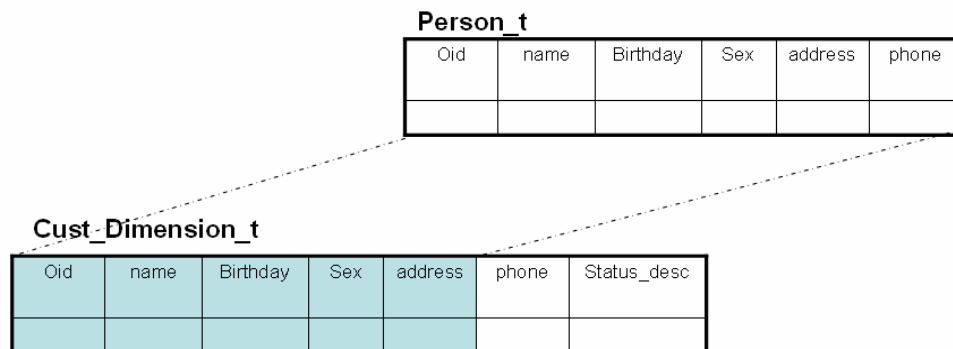
ต่อจากนี้ก็จะสร้างตารางของ Product ได้แล้ว

```
CREATE TABLE prod_dim_tab OF prod_dimension_t
```

```
(  
    REF IS OID USER GENERATED,  
    manufacturer WITH OPTION NOT NULL,  
    model_name WITH OPTION NOT NULL,  
    type WITH OPTION NOT NULL,  
    engine_size WITH OPTION NOT NULL  
)
```

4.1.3 Customer Dimension Table

สำหรับตารางไคเมนชันของลูกค้าและออปเจ็กไคเมนชันของลูกค้านั้นจะแตกต่างจาก Dimension Table และ Object Dimension ที่ผ่านมา ตรงที่โครงสร้างของ Customer Dimension นั้นสร้างมาจากโครงสร้างของข้อมูลที่มีชื่อ Cust_dimension_t ซึ่ง Cust_dimension_t นั้นจะมีโครงสร้างหลัก(Root Type) เป็น Person_t หมายความว่า Cust_dimension_t นั้นได้รับการถ่ายทอด (Inherit) คุณสมบัติ(Attribute) มาจาก Person_t ดังแสดงให้เห็นได้ในรูปที่ 5.4



รูปที่ 5.4 แสดงการถ่ายทอดคุณสมบัติระหว่าง Person_t กับ Cust_dimension_t

สำหรับ IBM DB2 นั้นไม่สามารถที่จะประกาศประเภทของแอทริบิวต์เป็น Complex Type ได้ ดังนั้นจึงทำให้เราต้องสร้างโครงสร้างข้อมูลชนิด Array ขึ้นมารองรับความสามารถของ Multivalue Attribute ดังรูปข้างต้น ต่อไปนี้เราจะแสดงคำสั่งในการสร้างโครงสร้างข้อมูล โดยเริ่มจากการประกาศ Composite Type ก่อน ดังคำสั่ง

```
CREATE TYPE name_t AS
(
    First VARCHAR(20),
    Middle VARCHAR(15),
    Last VARCHAR(20)
)
MODE DB2SQL
```

```
CREATE TYPE Address_t AS
(
    No INTEGER,
    Street VARCHAR(100),
    City VARCHAR (25),
    State VARCHAR (25),
    Zipcode VARCHAR (25)
)
MODE DB2SQL
```

ต่อมาเราก็จะแสดงคำสั่งในการสร้างมัลติแวลูเอททิปส์

```
CREATE TYPE phone_t AS  
(  
    location VARCHAR(15),  
    phone_num VARCHAR(12)  
)
```

MODE DB2SQL

```
CREATE TYPE phone_array_t AS  
(  
    phone1 PHONE_T,  
    phone2 PHONE_T,  
    phone3 PHONE_T,  
    phone4 PHONE_T  
)
```

MODE DB2SQL

REF USING INTEGER

```
CREATE TYPE status_desc_t AS  
(  
    Reg_id          CHAR(5),  
    reg_date        REF(time_dimension_t),  
    marital_status  VARCHAR(10),  
    year_income     INTEGER,  
    city            VARCHAR(25),  
    state           VARCHAR(25)  
)
```

MODE DB2SQL

```
CREATE TYPE status_desc_arr AS  
(  
    desc1  status_desc_t,  
    desc2  status_desc_t,  
    desc3  status_desc_t  
)
```

MODE DB2SQL

REF USING INTEGER

ต่อมาเราก็จะสร้างโครงสร้างหลัก ซึ่งก็คือ Person_t

```
CREATE TYPE person_t AS
(
    name          name_t,
    birthday      DATE,
    sex           CHAR(1),
    address       address_t,
    phone         phone_array_t
)
MODE DB2SQL
REF USING INTEGER
NOT FINAL
```

และสุดท้ายเราจะสร้าง Cust_dimension_t ที่ได้รับการถ่ายทอด มาจาก Person_t

```
CREATE TYPE cust_dimension_t UNDER person_t AS
(
    status_desc    status_desc_arr
)
MODE DB2SQL
```

เมื่อสร้างโครงสร้างข้อมูลทั้งหมดเรียบร้อยแล้วขั้นตอนต่อไปคือการสร้างตารางใดเมนชั้นของ
ลูกค้าที่ชื่อ Cust_dimension_tab เราจะใช้คำสั่งเอสคิวแอลในการสร้าง Cust_dimension_tab ดังนี้

```
CREATE TABLE cust_dim_tab OF cust_dimension_t
(
    REF IS OID USER GENERATED,
    name          WITH OPTIONS NOT NULL,
    birthday      WITH OPTIONS NOT NULL,
    sex           WITH OPTIONS NOT NULL,
    address       WITH OPTIONS NOT NULL,
    phone         WITH OPTIONS NOT NULL,
    status_desc   WITH OPTIONS NOT NULL
)
```

4.1.4 Sale Fact Table

เหมือนกับตารางอื่น ๆ ก็จะต้องสร้างโครงสร้างของออปเจ็ค ขึ้นมาก่อน ในที่นี้ก็คือ sale_fact_t ซึ่งในตารางแฟค นั้นจะมีแอททริบิวต์อยู่ 3 ตัวที่ทำหน้าที่เป็นพอยเตอร์ชี้ไปยังตารางอื่น ๆ ซึ่งทั้ง 3 ตัวนี้จะต้องมีทุกครั้งที่มีการเพิ่มแถวเข้าไปยังตาราง สำหรับคำสั่งที่ใช้ในการสร้าง Sale_Fact_tab นั้นแสดงได้ดังข้างล่างนี้

```
CREATE TYPE sale_fact_t AS
(
    cust_dim      REF(cust_dimension_t),
    product_dim   REF(prod_dimension_t),
    time_dim      REF(time_dimension_t),
    quantity      NUM,
    price         NUM
)
MODE DB2SQL
REF USING INTEGER
CREATE TABLE sale_fact_tab OF sale_fact_t
(
    REF IS OID USER GENERATED,
    cust_dim WITH OPTION NOT NULL,
    product_dim WITH OPTION NOT NULL,
    time_dim WITH OPTION NOT NULL,
    quantity WITH OPTION NOT NULL,
    price WITH OPTION NOT NULL
)
```

และในตาราง Sale_fact_tab นั้นมีแอททริบิวต์ 3 ตัวที่ทำหน้าที่เป็นพอยเตอร์ดังนั้นจึงต้องทำการ Add Scope เพื่อให้แอททริบิวต์ทั้ง 3 นั้นสามารถชี้ไปได้ถูกที่ สำหรับคำสั่งที่ใช้ในการเพิ่มมุมมอง(Add Scope)นั้นจะใช้คำสั่ง ALTER TABLE ดังคำสั่งข้างล่างนี้

```
ALTER TABLE sale_fact_tab ALTER COLUMN time_dim ADD SCOPE time_dim_tab
ALTER TABLE sale_fact_tab ALTER COLUMN product_dim ADD SCOPE prod_dim_tab
ALTER TABLE sale_fact_tab ALTER COLUMN cust_dim ADD SCOPE cust_dim_tab
```

4.2 ภาษาที่ใช้ในการเปลี่ยนแปลงข้อมูลในดาต้าแวร์เฮ้าส์(Data Manipulation Language)

โดยปกติแล้ว ดาต้าแวร์เฮ้าส์นั้นจะเก็บข้อมูลชนิดอ่านอย่างเดียว การเก็บข้อมูลลงดาต้าแวร์เฮ้าส์นั้นไม่จำเป็นมากนัก แต่อย่างไรก็ตามหากแอพพลิเคชั่นในตารางถูกเปลี่ยน ข้อมูลเก่าก็จะถูกแทนที่ด้วยข้อมูลใหม่ ที่เข้ามา หรือข้อมูลใหม่อาจจะถูกแทรกลงตารางโดยที่ข้อมูลเก่าไม่ได้ถูกเปลี่ยนแปลง สำหรับการลบข้อมูลนั้นแทบจะไม่จำเป็นในการใช้งานเลย เพราะว่าที่เก็บข้อมูล (Data Storage) มีราคาถูก แต่ถึงอย่างไรก็ตามการอัปเดต การลบ และการแทรกข้อมูลลงดาต้าแวร์เฮ้าส์ก็ยังคงต้องมีเพราะยังคงมีการเปลี่ยนแปลงข้อมูลในแต่ละไคเมนชันอยู่ สำหรับในหัวข้อนี้เราจะกล่าวถึง การแทรก การลบ และการอัปเดตข้อมูล

ในหัวข้อที่ผ่านมา เราแบ่งออกเป็น 4 หัวข้อย่อย แต่ละหัวข้อย่อยก็จะกล่าวถึงแต่ละตาราง สำหรับในหัวข้อนี้เราจะพูดถึงภาษาที่ใช้ในการเปลี่ยนแปลงข้อมูลไปที่ตาราง

4.2.1 Time Dimension Table

สำหรับคำสั่งที่ใช้ในการแทรกข้อมูลลงในตาราง time_dim_tab นั้น จะแสดงให้เห็นดังข้างล่างนี้

```
INSERT INTO time_dim_tab VALUES (  
time_dimension_t(0),'1999Q1M01',1999,'Quarter1','January');
```

การอัปเดตข้อมูลสามารถทำได้โดยการใช้คำสั่ง

```
UPDATE time_dim_tab  
SET fiscal_year = 1998,  
fiscal_qtr='Quarter2',  
fiscal_month = 'April'  
WHERE time_key = '1999Q1M01'
```

การลบข้อมูลสามารถทำได้โดยการใช้คำสั่ง

```
DELETE FROM time_dim_tab  
WHERE time_key = '1999Q1M01'
```

จากการศึกษาเราจะพบว่า คีย์ที่เรานำมาใช้ในการอ้างอิง (time_key : อาจจะมีหรือไม่ก็ได้เพราะยังไม่มี OID อยู่แล้ว แต่ในที่นี้ให้มีเพราะเพื่อให้ง่ายต่อการอ้างอิง) นั้นเป็นคีย์ที่เป็นลักษณะคีย์ที่มีความหมาย (Meaningful values) ข้อดีของการใช้คีย์ที่มีความหมายคือการที่ไม่ต้องประยุกต์ใช้ในการตอบคำถาม แต่ในทางปฏิบัติแล้วการใช้คีย์ที่มีความหมายเป็นสิ่งที่ควรหลีกเลี่ยงเพราะหากต้องมีการเปลี่ยนแปลงค่าแล้ว จะทำให้คีย์จะต้องถูกเปลี่ยนด้วยซึ่งเป็นสิ่งที่ไม่สมควรทำอย่างยิ่ง

4.2.2 Product Dimension

สำหรับคำสั่งในการ Insert Delete Update ข้อมูลใน prod_dim_tab นั้นจะมีลักษณะง่าย ๆ คล้ายกับคำสั่งที่ใช้ในการ Insert Delete Data ข้อมูลใน time_dim_tab ซึ่งได้กล่าวไปแล้วข้างต้น สำหรับคำสั่งในการแทรกข้อมูลลงใน prod_dim_tab นั้นแสดงได้ดังนี้

```
INSERT INTO prod_dim_tab VALUES(  
    prod_dimension_t(0),  
    'Corolla',  
    'Toyota',  
    'CE 4dr Seden(4cyl,4A)',  
    '1800CC')
```

ข้อมูลสามารถอัปเดตได้โดยการใช้คำสั่งดังนี้

```
UPDATE prod_dim_tab  
SET model_name = 'Civic',  
    Type = '2 Door Sedan',  
    Engine_size = '500CC'  
WHERE model_name = 'Corolla'
```

คำสั่งข้างล่างนี้เป็นคำสั่งที่ใช้ในการลบ

```
DELETE FROM prod_dim_tab  
WHERE model_name = 'Civic', Type = '2 Door Sedan'
```

4.2.3 ตารางไคเมนชันของลูกค้า

สำหรับไคเมนชันนี้จะแตกต่างจากไคเมนชันอื่น เนื่องจากไคเมนชันของลูกค้านั้นจะประกอบไปด้วยข้อมูลประเภทโครงสร้างซึ่งได้แก่ข้อมูลประเภท คอมพิวเตอร์แอททริบิวต์ , มัลติแวลูแอททริบิวต์ ซึ่งภาษาที่ใช้ในการเปลี่ยนแปลงข้อมูลในดาต้าแวร์เฮ้าส์ก็จะมีลักษณะที่แตกต่างออกไป ดังตัวอย่างข้างล่างนี้ ก็จะเป็นการแทรกข้อมูลลงในตารางไคเมนชันของลูกค้า ที่ชื่อ cust_dim_tab

```
INSERT INTO cust_dim_tab VALUES (  
    cust_dimension_t(0),  
    name_t()..First('Jonathan')..Middle('-')..Last('Abbott'),  
    '12/11/1999',  
    'M',  
    Address_t()..Street('142 S Sheridan Road')..City('Tulsa')..State('Oklahoma')..Zipcode('74112')  
    ,phone_array_t()  
    ..phone1(phone_t()..Location('Home')..phone_num('918-835-3161'))  
    ..phone2(phone_t()..Location('Mobile')..Phone_num('338-331-4463'))
```



```

..phone3(phone_t()..Location(")..phone_num(""))
..phone4(phone_t()..Location(")..phone_num(""))
,status_desc_arr()
..desc1(status_desc_t()
  ..Reg_id('R01')
  ..reg_date((select oid from time_dim_tab where time_key = '1999Q3M09'))
  ..marital_status('Unmarried')
  ..year_income(60000)
  ..city('Tulsa')
  ..state('Oklahoma'))
)

```

สำหรับคำสั่งที่ใช้ในการอัปเดตข้อมูลนั้นเนื่องจากว่า IBM DB2 ยังไม่สนับสนุนการทำงานใน
รูปแบบข้อมูลประเภท Structure Type นัก ทำให้การอัปเดตข้อมูลในตารางที่มีข้อมูลประเภท Composite
อยู่นั้นจำเป็นต้องเอาข้อมูลเก่าเก็บลงไปด้วย เพื่อไม่ให้ข้อมูลเก่าหายดังแสดงได้ในตัวอย่างข้างล่างนี้

```

UPDATE cust_dim_tab
SET status_desc = (status_desc_arr()
..desc1(status_desc_t()
  ..Reg_id('R01')
  ..reg_date((select oid from time_dim_tab where time_key = '1999Q3M09'))
  ..marital_status('Unmarried')
  ..year_income(60000)
  ..city('Tulsa')
  ..state('Oklahoma'))
..desc2(status_desc_t()
  ..Reg_id('R02')
  ..Reg_date((select oid from time_dim_tab where time_key = '2001Q1M01'))
  ..marital_status('Married')
  ..Year_income(66000)
  ..city('Skokie')
  ..state('Illinolis'))
..desc3(status_desc_t()
  ..Reg_id('R03')
  ..Reg_date((select oid from time_dim_tab where time_key = '2002Q1M03'))
  ..marital_status('Married')
  ..Year_income(70000)
  ..city('Tulsa')
  ..state('Oklahoma'))
)

```

```
)  
WHERE name..first = 'Jonathan'
```

คำสั่งสุดท้ายเป็นคำสั่งที่เกี่ยวข้องกับการลบข้อมูลในตาราง แสดงให้เห็นได้ดังนี้

```
DELETE FROM cust_dim_tab where name.first = 'Jonathan'
```

5.5.5 ตารางแสดงข้อมูลการขายสินค้า

แอททริบิวต์ของตารางแฟคตนั้นจะเป็นแอททริบิวต์ง่าย ๆ ไม่ซับซ้อน สำหรับการแทรกข้อมูลลงตารางนั้นเราจะใช้คำสั่งดังต่อไปนี้

```
INSERT INTO sale_fact_tab VALUES(  
sale_fact_t(0),  
(select oid from cust_dim_tab where name..first = 'Jonathan'),  
(select oid from prod_dim_tab where model_name = 'CR-V'),  
(select oid from time_dim_tab where time_key = '1999Q3M09'),  
1,21550)
```